

Consultas SQL para el análisis exploratorio

Vamos a analizar los datos mediante algunas consultas de SQL. Empezamos importando las librerías necesarias y accediendo a la base de datos de Aiven.

```
In [1]: # !pip install jupysql pymysql sqlalchemy python-dotenv
```

```
In [2]: %load_ext sql
```

```
In [3]: import os
import urllib.parse
from dotenv import load_dotenv

dotenv_path = "/Users/crganfornina/Documents/Máster Data Science/TFM/.venv/"
load_dotenv(dotenv_path)

USER = os.getenv('DB_USER', 'avnadmin')
PASSWORD = os.getenv('DB_PASSWORD')
HOST = os.getenv('DB_HOST')
PORT = os.getenv('DB_PORT', '10854')
DATABASE = os.getenv('DB_DATABASE', 'imdb_tfm')
SAFE_PASSWORD = urllib.parse.quote_plus(PASSWORD) if PASSWORD else ""

os.environ['DATABASE_URL'] = f"mysql+pymysql://{USER}:{SAFE_PASSWORD}@{HOST}"

%sql
```

```
In [4]: %%sql
USE
imdb_tfm;
```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

```
Out [4]:
```

Análisis de evolución histórica y de géneros

El objetivo de este bloque es entender los patrones del mercado a lo largo de la historia. Vamos a analizar las tendencias de producción, cómo ha evolucionado el volumen de obras por décadas y si existe alguna relación entre la duración de las producciones y la popularidad entre el público.

Vamos a empezar analizando la **evolución del volumen de la producción** a lo largo de la historia: ¿cómo ha crecido la producción del contenido por década y tipo de obra, y a qué ritmo avanza la industria año tras año (YoY)?

Producción y volumen acumulado por década y tipo de obra

Agrupamos los títulos por década y calculamos el número de títulos por formato, además de la media de calificación y el total de votos. Esto nos va a permitir ver qué tipo de formatos han sido más populares cada época de la historia.

```
In [5]: %%sql
SELECT
    FLOOR(f.Año / 10) * 10 AS Decada,
    t.titleType AS TipoTitulo,
    COUNT(t.titleID) AS TotalTitulos,
    ROUND(AVG(r.averageRating), 2) AS CalificacionMedia,
    SUM(r.numVotes) AS VotosTotales
FROM DIM_TITLES t
INNER JOIN DIM_FECHA f ON t.startYear = f.FechaID
LEFT JOIN H_RATINGS r ON t.titleID = r.titleID
GROUP BY FLOOR(f.Año / 10) * 10, t.titleType
ORDER BY Decada DESC, TotalTitulos DESC;
```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-
dbds.i.aivencloud.com:10854/imdb_tfm'
101 rows affected.

```
Out[5]:
```

Decada	TipoTitulo	TotalTitulos	CalificacionMedia	VotosTotales
2020	tvEpisode	69806	7.60	53531317
2020	movie	34194	5.95	145566645
2020	tvSeries	12728	6.80	40095996
2020	tvMiniSeries	4479	7.22	15196594
2020	short	3316	6.79	1200629
2020	tvMovie	2987	6.06	1746622
2020	tvSpecial	1316	6.69	1086416
2020	videoGame	1103	7.07	755848
2020	video	652	7.28	582478
2020	tvShort	31	6.81	38480

Truncated to *displaylimit* of 10.

En la década actual podemos ver que se producen con diferencia muchos más episodios de series que otros tipos de producciones. Además, la calificación media de este tipo de producción es la más alta de entre todos los tipos. Por otro lado, con respecto a votos totales ganan las películas, a pesar de tener la mitad de producción.

Crecimiento Año tras Año (YoY) de Películas

Ahora vamos a ver cómo crece la producción tanto general como de películas año tras año.

```
In [6]: %sql
WITH ProduccionAnual AS (
    SELECT
        f.Año,
        COUNT(t.titleID) AS TotalTitulos,
        COUNT( CASE WHEN t.titleType = 'movie' THEN 1 END) AS TotalSoloPelic
    FROM DIM_TITLES t
    INNER JOIN DIM_FECHA f ON t.startYear = f.FechaID
    -- WHERE t.titleType = 'movie'
    GROUP BY f.Año
)
SELECT
    Año,
    TotalTitulos,
    LAG(TotalTitulos, 1) OVER (ORDER BY Año) AS TitulosAnioAnterior,
    (TotalTitulos - LAG(TotalTitulos, 1) OVER (ORDER BY Año)) AS VariacionAn
    ROUND(
        ((TotalTitulos - LAG(TotalTitulos, 1) OVER (ORDER BY Año)) /
        LAG(TotalTitulos, 1) OVER (ORDER BY Año)) * 100,
        2
    ) AS CrecimientoYoY_Titulos,
    TotalSoloPeliculas,
    LAG(TotalSoloPeliculas, 1) OVER (ORDER BY Año) AS PeliculasAnioAnterior,
    (TotalSoloPeliculas - LAG(TotalSoloPeliculas, 1) OVER (ORDER BY Año)) AS
    ROUND(
        ((TotalSoloPeliculas - LAG(TotalSoloPeliculas, 1) OVER (ORDER BY Año)
        LAG(TotalSoloPeliculas, 1) OVER (ORDER BY Año)) * 100,
        2
    ) AS CrecimientoYoY_Peliculas
FROM ProduccionAnual
WHERE Año IS NOT NULL
ORDER BY Año DESC;
```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-
dbds.i.aivencloud.com:10854/imdb_tfm'

146 rows affected.

Out [6]:

Año	TotalTitulos	TitulosAnioAnterior	VariacionAbsolutaTitulos	CrecimientoYoY_Titulo
2026	893	17667	-16774	-94.9
2025	17667	20893	-3226	-15.4
2024	20893	23189	-2296	-9.9
2023	23189	23972	-783	-3.2
2022	23972	23133	839	3.6
2021	23133	20865	2268	10.8
2020	20865	23385	-2520	-10.7
2019	23385	22132	1253	5.6
2018	22132	21521	611	2.8
2017	21521	20221	1300	6.4

Truncated to `displaylimit` of 10.

Podemos ver la evolución de la industria en los últimos años. En concreto, podemos destacar que hubo una caída importante en 2020, como resultado del impacto de la pandemia. Además, estos últimos años también ha habido una disminución de títulos.

Vamos a analizar ahora el **éxito de los diferentes géneros cinematográficos**, es decir, ¿cuáles son los géneros más producidos cada década y cómo reacciona la gente ante estos?

Top 5 Géneros más producidos por década

In [7]:

```

%%sql
WITH GenerosPorDecada AS (
    SELECT
        FLOOR(f.Año / 10) * 10 AS Decada,
        g.genreName AS Genero,
        COUNT(t.titleID) AS CantidadTitulos
    FROM DIM_TITLES t
    INNER JOIN DIM_FECHA f ON t.startYear = f.FechaID
    INNER JOIN H_TITLES_GENRES hg ON t.titleID = hg.titleID
    INNER JOIN DIM_GENRES g ON hg.genreID = g.genreID
    GROUP BY FLOOR(f.Año / 10) * 10, g.genreName
),
RankingsGeneros AS (
    SELECT
        Decada,
        Genero,
        CantidadTitulos,
        DENSE_RANK() OVER (PARTITION BY Decada ORDER BY CantidadTitulos DESC) AS Posicion
    FROM GenerosPorDecada
)
SELECT Decada, Genero, CantidadTitulos, Posicion
FROM RankingsGeneros

```

```
WHERE Posicion <= 5 AND Decada IS NOT NULL
ORDER BY Decada DESC, Posicion ASC;
```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

79 rows affected.

Out [7]:

Decada	Genero	CantidadTitulos	Posicion
--------	--------	-----------------	----------

2020	Drama	60088	1
2020	Comedy	36535	2
2020	Action	23574	3
2020	Crime	20456	4
2020	Animation	18767	5
2010	Drama	81874	1
2010	Comedy	63437	2
2010	Action	33910	3
2010	Adventure	27468	4
2010	Crime	26612	5

Truncated to [displaylimit](#) of 10.

Géneros que superaron la media histórica de su propio género

Vamos a ver en qué épocas destacaron los distintos géneros, es decir, cuándo superaron su media histórica.

```
In [8]: %%sql
WITH MediaHistoricaGeneros AS (
    SELECT
        g.genreID,
        g.genreName AS Genero,
        AVG(r.averageRating) AS RatingMedioHistorico
    FROM H_TITLES_GENRES hg
    INNER JOIN DIM_GENRES g ON hg.genreID = g.genreID
    INNER JOIN H_RATINGS r ON hg.titleID = r.titleID
    GROUP BY g.genreID, g.genreName
)
SELECT
    FLOOR(f.Año / 10) * 10 AS Decada,
    g.genreName AS Genero,
    ROUND(AVG(r.averageRating), 2) AS RatingMedioDecada,
    ROUND(mh.RatingMedioHistorico, 2) AS RatingMedioHistorico,
    ROUND(AVG(r.averageRating) - mh.RatingMedioHistorico, 2) AS DesviacionPc
FROM DIM_TITLES t
INNER JOIN DIM_FECHA f ON t.startYear = f.FechaID
INNER JOIN H_RATINGS r ON t.titleID = r.titleID
INNER JOIN H_TITLES_GENRES hg ON t.titleID = hg.titleID
INNER JOIN DIM_GENRES g ON hg.genreID = g.genreID
```

```
INNER JOIN MediaHistoricaGeneros mh ON g.genreID = mh.genreID
GROUP BY FLOOR(f.Año / 10) * 10, g.genreName, mh.RatingMedioHistorico
HAVING RatingMedioDecada > mh.RatingMedioHistorico AND COUNT(t.titleID) >= 1
ORDER BY DesviacionPositiva DESC, Decada DESC;
```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

87 rows affected.

Out [8]:

Decada	Genero	RatingMedioDecada	RatingMedioHistorico	DesviacionPositiva
1960	Horror	6.66	5.67	0.99
1960	Western	7.43	6.95	0.48
2020	Short	6.94	6.48	0.45
1930	Horror	6.07	5.67	0.40
1960	Thriller	6.51	6.14	0.37
1950	Western	7.32	6.95	0.36
2010	Musical	6.85	6.51	0.34
1980	Family	7.27	6.96	0.32
1950	Thriller	6.46	6.14	0.32
1950	Horror	5.99	5.67	0.32

Truncated to *displaylimit* of 10.

Índice de Discrepancia: Interacción del público frente a la crítica

Vamos a ver ahora una comparación entre las notas de cada género y los votos que acumulan por título. El índice de discrepancia lo calcularemos como la diferencia entre los dos rankings. Un valor positivo muy alto significa que es un género muy comercial, es decir, vota mucha gente pero las notas son normales. Un valor negativo, por otro lado, significa que lo ven muy pocas personas, pero las que lo hacen le ponen muy buenas notas.

In [9]:

```
%%sql
WITH MetricasGeneros AS (
    SELECT
        g.genreName AS Genero,
        COUNT(t.titleID) AS TotalTitulos,
        ROUND(AVG(r.averageRating), 2) AS RatingMedio,
        ROUND(AVG(r.numVotes), 0) AS VotosMediosPorTitulo,
        SUM(r.numVotes) AS VotosTotales
    FROM H_TITLES_GENRES hg
    INNER JOIN DIM_GENRES g ON hg.genreID = g.genreID
    INNER JOIN H_RATINGS r ON hg.titleID = r.titleID
    INNER JOIN DIM_TITLES t ON hg.titleID = t.titleID
    GROUP BY g.genreID, g.genreName
    HAVING COUNT(t.titleID) >= 200 -- Filtramos géneros muy minoritarios par
```

```

),
Rankings AS (
    SELECT
        Genero,
        TotalTitulos,
        RatingMedio,
        VotosMediosPorTitulo,
        VotosTotales,
        -- Rankeamos los géneros según la crítica (mejor nota = mejor puesto)
        DENSE_RANK() OVER (ORDER BY RatingMedio DESC) AS RankCritica,
        -- Rankeamos los géneros según el engagement del público (más votos = mejor puesto)
        DENSE_RANK() OVER (ORDER BY VotosMediosPorTitulo DESC) AS RankInteraccion
    FROM MetricasGeneros
)
SELECT
    Genero,
    TotalTitulos,
    RatingMedio,
    RankCritica,
    VotosMediosPorTitulo,
    RankInteraccion,
    -- Discrepancia:
    -- Un valor muy positivo indica que el público vota mucho pero mal
    -- Un valor muy negativo indica que la gente vota muy bien pero poco
    (CAST(RankCritica AS SIGNED) - CAST(RankInteraccion AS SIGNED)) AS IndiceDiscrepancia
CASE
    WHEN (CAST(RankCritica AS SIGNED) - CAST(RankInteraccion AS SIGNED)) > 10
    THEN 'Populares (Muchos votos, nota regular)'
    WHEN (CAST(RankCritica AS SIGNED) - CAST(RankInteraccion AS SIGNED)) < -10
    THEN 'Obras de Culto (Pocos votos, excelente nota)'
    ELSE 'Equilibrio (Coherencia entre votos y nota)'
END AS ClasificacionAudiencia
FROM Rankings
ORDER BY IndiceDiscrepancia DESC;

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

28 rows affected.

Out [9]:

Genero	TotalTitulos	RatingMedio	RankCritica	VotosMediosPorTitulo	RankInteracci
--------	--------------	-------------	-------------	----------------------	---------------

Thriller	38523	6.14	24	6388	
Sci-Fi	18685	6.41	22	9530	
Horror	34936	5.67	25	4610	
Film-Noir	866	6.47	21	4755	
Fantasy	31077	6.96	13	5033	
Biography	15269	7.07	8	5960	
Romance	60773	6.82	17	3099	
Action	105886	7.06	9	5009	
Musical	6021	6.51	19	2246	
Comedy	198914	6.91	16	2543	

Truncated to *displaylimit* of 10.

Vamos a analizar ahora la **relación entre la duración de las películas y sus calificaciones**, es decir, ¿las películas más largas les gustan más al público o aburren más?

Análisis de calificaciones por rangos de duración

In [10]:

```
%%sql
SELECT
  CASE
    WHEN t.runtimeMinutes < 60 THEN 'Cortometraje (< 1h)'
    WHEN t.runtimeMinutes BETWEEN 60 AND 120 THEN 'Duración Comercial Es
    WHEN t.runtimeMinutes BETWEEN 121 AND 180 THEN 'Película Larga (2h -
    ELSE 'Súper Producción (> 3h)'
  END AS RangoDuracion,
  COUNT(t.titleID) AS CantidadPeliculas,
  ROUND(AVG(r.averageRating), 2) AS NotaMedia,
  ROUND(AVG(r.numVotes), 0) AS VotosMedios,
  ROUND(MAX(r.averageRating), 1) AS CalificacionMaxima
FROM DIM_TITLES t
INNER JOIN H_RATINGS r ON t.titleID = r.titleID
WHERE t.titleType = 'movie' AND t.runtimeMinutes IS NOT NULL
GROUP BY
  CASE
    WHEN t.runtimeMinutes < 60 THEN 'Cortometraje (< 1h)'
```



```

        WHEN t.runtimeMinutes BETWEEN 60 AND 120 THEN 'Duración Comercial Es
        WHEN t.runtimeMinutes BETWEEN 121 AND 180 THEN 'Película Larga (2h -
        ELSE 'Súper Producción (> 3h)'
    END
ORDER BY RangoDuracion ASC;

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

4 rows affected.

Out [10]:

RangoDuracion	CantidadPelículas	NotaMedia	VotosMedios	CalificacionMaxima
Cortometraje (< 1h)	4725	6.35	374	9.8
Duración Comercial Estándar (1h - 2h)	154804	5.89	5062	9.9
Película Larga (2h - 3h)	20948	6.51	21154	10.0
Súper Producción (> 3h)	987	7.03	21384	9.5

Podemos ver que a mayor duración, mejor nota media.

El factor humano en el éxito de las producciones

En este segundo bloque vamos a centrarnos en las personas, concretamente en los actores, actrices y directores. Así, vamos a ver quiénes son los profesionales con más éxito y a qué edades suelen hacer sus mejores trabajos.

Vamos a empezar analizando la **calidad de los profesionales**, es decir, ¿qué profesionales están más asociados a producciones con valoraciones excelentes?

Profesionales con más películas por encima del 8.0

Buscamos a los profesionales con películas calificadas con 8 o más, y que además tengan al menos 100000 votos en IMDb, para garantizar que estos títulos sean conocidos.

```

In [11]: %%sql
WITH ProfesionalesExcelentes AS (
    SELECT
        p.nameID,
        p.category AS Rol,
        COUNT(t.titleID) AS PeliculasExcelentes
    FROM H_PRINCIPALS p
    INNER JOIN DIM_TITLES t ON p.titleID = t.titleID
    INNER JOIN H_RATINGS r ON t.titleID = r.titleID
    WHERE r.averageRating >= 8.0 AND r.numVotes >= 100000
    AND p.category IN ('actor', 'actress', 'director')

```

```

        GROUP BY p.nameID, p.category
    )
    SELECT
        n.primaryName AS Nombre,
        pe.Rol,
        pe.PeliculasExcelentes,
        n.primaryProfession AS Profesion
    FROM ProfesionalesExcelentes pe
    INNER JOIN DIM_NAMES n ON pe.nameID = n.nameID AND pe.Rol = n.primaryProfession
    ORDER BY pe.PeliculasExcelentes DESC, Nombre ASC

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

4806 rows affected.

Out[11]:

	Nombre	Rol	PeliculasExcelentes	Profesion
	Dee Bradley Baker	actor	16	actor
	J.K. Simmons	actor	10	actor
	Harrison Ford	actor	9	actor
	John DiMaggio	actor	9	actor
	Leonardo DiCaprio	actor	9	actor
	Mark Ruffalo	actor	9	actor
	Mayumi Tanaka	actress	9	actress
	Robert De Niro	actor	9	actor
	Tom Kenny	actor	9	actor
	Aamir Khan	actor	8	actor

Truncated to [displaylimit](#) of 10.

Vamos a ver ahora **a qué edad tienen los artistas su mayor éxito**, es decir, ¿a qué edad suelen alcanzar los profesionales su mejor momento?

Calificación promedio de películas según la edad del profesional en el año de estreno

In [12]:

```

%%sql
SELECT
    n.primaryProfession AS RolProfesional,
    CASE
        WHEN (f.Año - n.birthYear) < 25 THEN '<25'
        WHEN (f.Año - n.birthYear) BETWEEN 25 AND 34 THEN '25-34'
        WHEN (f.Año - n.birthYear) BETWEEN 35 AND 44 THEN '35-44'
        WHEN (f.Año - n.birthYear) BETWEEN 45 AND 54 THEN '45-54'
        WHEN (f.Año - n.birthYear) BETWEEN 55 AND 64 THEN '55-64'
        ELSE '65+'
    END AS RangoEdad,
    COUNT(DISTINCT t.titleID) AS TotalProducciones,
    ROUND(AVG(r.averageRating), 2) AS CalificacionMedia,

```

```

ROUND(AVG(r.numVotes), 0) AS EngagementMedio
FROM DIM_TITLES t
INNER JOIN DIM_FECHA f ON t.startYear = f.FechaID
INNER JOIN H_RATINGS r ON t.titleID = r.titleID
INNER JOIN H_PRINCIPALS p ON t.titleID = p.titleID
INNER JOIN DIM_NAMES n ON p.nameID = n.nameID AND p.category = n.primaryProf
WHERE n.birthYear IS NOT NULL
AND f.Año IS NOT NULL
AND p.category IN ('actor', 'actress', 'director')
AND r.numVotes >= 5000
GROUP BY
    n.primaryProfession,
    CASE
        WHEN (f.Año - n.birthYear) < 25 THEN '<25'
        WHEN (f.Año - n.birthYear) BETWEEN 25 AND 34 THEN '25-34'
        WHEN (f.Año - n.birthYear) BETWEEN 35 AND 44 THEN '35-44'
        WHEN (f.Año - n.birthYear) BETWEEN 45 AND 54 THEN '45-54'
        WHEN (f.Año - n.birthYear) BETWEEN 55 AND 64 THEN '55-64'
        ELSE '65+'
    END
ORDER BY RolProfesional, RangoEdad;

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

18 rows affected.

Out[12]:

RolProfesional	RangoEdad	TotalProducciones	CalificacionMedia	EngagementMedio
actor	<25	7945	6.96	56792
actor	25-34	17930	7.15	53444
actor	35-44	20622	7.11	55113
actor	45-54	18451	7.08	55149
actor	55-64	12736	6.97	52752
actor	65+	8560	6.99	57815
actress	<25	9669	6.94	50935
actress	25-34	17302	7.09	47848
actress	35-44	13265	7.10	47740
actress	45-54	8451	7.10	43773

Truncated to *displaylimit* of 10.

Podemos ver que para actores y actrices su mejor etapa es de los 25 a los 44 años, donde producen más títulos de éxito y acumulan más votos.

Análisis del género de la televisión y las series

La estructura de las series es muy distinta a la del cine. En este bloque vamos a analizar las series para entender cómo se comportan temporada tras temporada.

Vamos a empezar comparando el formato de películas individuales (`movie`) con el de episodios de series (`tvEpisode`), es decir, ¿qué **formato consigue generar más fidelidad** en los espectadores?

Comparativa estadística descriptiva por formato

Vamos a hacer una comparación entre películas y episodios, calculando la cantidad total de títulos, la nota media, el promedio de votos que se lleva cada formato y la desviación típica.

```
In [13]: %%sql
SELECT
    t.titleType AS Formato,
    COUNT(t.titleID) AS VolumenTitulos,
    ROUND(AVG(r.averageRating), 2) AS CalificacionMedia,
    ROUND(AVG(r.numVotes), 0) AS VotosPromedio,
    ROUND(STDDEV_SAMP(r.averageRating), 2) AS DesviacionTipicaRating,
    SUM(r.numVotes) AS VotosTotales
FROM DIM_TITLES t
INNER JOIN H_RATINGS r ON t.titleID = r.titleID
WHERE t.titleType IN ('movie', 'tvEpisode')
GROUP BY t.titleType;
```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

2 rows affected.

```
Out[13]:
```

Formato	VolumenTitulos	CalificacionMedia	VotosPromedio	DesviacionTipicaRating
movie	186719	5.98	6697	1.28
tvEpisode	264964	7.61	682	0.91

Aunque las películas individuales acumulan muchos más votos (porque se ven de una vez), los episodios de las series tienen una nota media mucho más alta con diferencia, y una desviación mucho más estrecha.

Ahora, vamos a ver el **comportamiento de las series a lo largo del tiempo**, es decir, ¿van perdiendo calidad las series de televisión a medida que avanzan las temporadas?

Evolución de la calidad entre temporadas

Vamos a calcular la nota media que tiene cada temporada de todas las series, y vamos a comparar la nota media de una temporada con su temporada anterior para sacar la diferencia de calidad. Así, vemos si las series pierden éxito con el paso de los años o si por el contrario mejoran.

```

In [14]: %%sql
WITH NotaMediaTemporada AS (
    SELECT
        serie.primaryTitle AS NombreSerie,
        ep.parentTitleID AS SerieID,
        ep.seasonNumber AS Temporada,
        AVG(r.averageRating) AS RatingMedioTemporada,
        SUM(r.numVotes) AS VotosTemporada
    FROM DIM_TITLES ep
    INNER JOIN DIM_TITLES serie ON ep.parentTitleID = serie.titleID
    INNER JOIN H_RATINGS r ON ep.titleID = r.titleID
    WHERE ep.titleType = 'tvEpisode' AND ep.seasonNumber IS NOT NULL
    GROUP BY serie.primaryTitle, ep.parentTitleID, ep.seasonNumber
),
VotosTotales AS (
    SELECT
        NombreSerie,
        SerieID,
        Temporada,
        RatingMedioTemporada,
        SUM(VotosTemporada) OVER (PARTITION BY SerieID) AS VotosTotalesSerie,
        LAG(RatingMedioTemporada, 1) OVER (PARTITION BY SerieID ORDER BY Temporada) AS RatingTemporadaAnterior,
    FROM NotaMediaTemporada
),
CalculoRanking AS (
    SELECT
        NombreSerie,
        Temporada,
        ROUND(RatingMedioTemporada, 2) AS RatingActual,
        ROUND(RatingTemporadaAnterior, 2) AS RatingTemporadaAnterior,
        ROUND(RatingMedioTemporada - RatingTemporadaAnterior, 2) AS DiferenciaDeCalidad,
        DENSE_RANK() OVER (ORDER BY VotosTotalesSerie DESC) AS RankingPopularidad
    FROM VotosTotales
)
SELECT
    RankingPopularidad,
    NombreSerie,
    Temporada,
    RatingActual,
    RatingTemporadaAnterior,
    DiferenciaDeCalidad
FROM CalculoRanking
WHERE Temporada >= 2
ORDER BY RankingPopularidad ASC, Temporada ASC;

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

13218 rows affected.

Out [14]: **Ranking****Popularidad** **NombreSerie** **Temporada** **RatingActual** **RatingTemporadaAnterior**

1	Game of Thrones	2	8.81	9.00
1	Game of Thrones	3	8.93	8.81
1	Game of Thrones	4	9.24	8.93
1	Game of Thrones	5	8.70	9.24
1	Game of Thrones	6	8.98	8.70
1	Game of Thrones	7	9.03	8.98
1	Game of Thrones	8	6.40	9.03
3	Attack on Titan	2	9.09	8.81
3	Attack on Titan	3	9.30	9.09
3	Attack on Titan	4	9.23	9.30

Truncated to *displaylimit* of 10.

Vamos ahora a identificar los **episodios clave de cada serie**, es decir, el mejor capítulo, el peor y si hay alguno que destaque por encima de todos los demás.

Extracción del mejor y el peor capítulo de cada serie

```
In [15]: %%sql
WITH Episodios AS (
    SELECT
        serie.primaryTitle AS NombreSerie,
        ep.seasonNumber AS Temporada,
        ep.episodeNumber AS NumeroEpisodio,
        ep.primaryTitle AS TituloEpisodio,
        r.averageRating AS RatingEpisodio,
        r.numVotes AS VotosEpisodio,
        ROW_NUMBER() OVER (PARTITION BY ep.parentTitleID ORDER BY r.averageF
        ROW_NUMBER() OVER (PARTITION BY ep.parentTitleID ORDER BY r.averageF
    FROM DIM_TITLES ep
    INNER JOIN DIM_TITLES serie ON ep.parentTitleID = serie.titleID
    INNER JOIN H_RATINGS r ON ep.titleID = r.titleID
    WHERE ep.titleType = 'tvEpisode'
)
SELECT
    NombreSerie,
    Temporada,
```

```

NumeroEpisodio,
TituloEpisodio,
RatingEpisodio,
VotosEpisodio,
CASE
    WHEN RnkMejor = 1 THEN 'MEJOR EPISODIO'
    WHEN RnkPeor = 1 THEN 'PEOR EPISODIO'
END AS TipoHito
FROM Episodios
WHERE RnkMejor = 1 OR RnkPeor = 1
ORDER BY NombreSerie ASC, TipoHito DESC;

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-bds.i.aivencloud.com:10854/imdb_tfm'

24076 rows affected.

Out[15]:

NombreSerie	Temporada	NumeroEpisodio	TituloEpisodio	RatingEpisodio	VotosEpisodio
:Dryvrs	1	1	Just Me in the House by Myself	7.8	
¿Qué fue de Jorge Sanz?	3	1	¿Qué fue de Jorge Sanz? Buena racha	6.9	
¿Qué fue de Jorge Sanz?	2	1	¿Qué fue de Jorge Sanz? 5 años después	7.2	
¿Quién lo mató?	1	1	Jorge	6.4	
¿Quién lo mató?	1	5	Paola	7.4	
.hack//SIGN	1	1	Role Play	6.7	
.hack//SIGN	1	2	Guardian	6.8	
'Allo 'Allo!	7	1	A Quiet Honeymoon	7.5	2
'Allo 'Allo!	2	7	The Gâteau from the Château	8.7	3
'Til Death	4	21	The Wedding	5.6	

Truncated to [displaylimit](#) of 10.

Episodios clave (desviaciones notables sobre el promedio de la serie)

Vamos a identificar los capítulos individuales que tengan una calificación de 2 puntos o más por encima de la media de la misma serie.

```

In [16]: %%sql
WITH PromedioSeries AS (
    SELECT
        parentTitleID AS SerieID,
        AVG(r.averageRating) AS PromedioGlobalSerie,
        COUNT(ep.titleID) AS CantidadEpisodios
    FROM DIM_TITLES ep
    INNER JOIN H_RATINGS r ON ep.titleID = r.titleID
    WHERE ep.titleType = 'tvEpisode'
    GROUP BY parentTitleID
    HAVING CantidadEpisodios >= 10
)
SELECT
    serie.primaryTitle AS NombreSerie,
    ep.seasonNumber AS Temporada,
    ep.episodeNumber AS NumeroEpisodio,
    ep.primaryTitle AS TituloEpisodio,
    r.averageRating AS NotaEpisodio,
    ROUND(ps.PromedioGlobalSerie, 2) AS NotaMediaSerie,
    ROUND(r.averageRating - ps.PromedioGlobalSerie, 2) AS DesviacionPositiva
FROM DIM_TITLES ep
INNER JOIN DIM_TITLES serie ON ep.parentTitleID = serie.titleID
INNER JOIN H_RATINGS r ON ep.titleID = r.titleID
INNER JOIN PromedioSeries ps ON ep.parentTitleID = ps.SerieID
WHERE r.averageRating > (ps.PromedioGlobalSerie + 2.0) -- Desviación de +2 p
ORDER BY DesviacionPositiva DESC;

```

Running query in 'mysql+pymysql://avnadmin:***@imdb-tfm-cr-dbds.i.aivencloud.com:10854/imdb_tfm'

419 rows affected.

Out [16]:	NombreSerie	Temporada	NumeroEpisodio	TituloEpisodio	NotaEpisodio	NotaMediaSe
	Gando	2	16	Episode #2.16	8.7	3
	The Tonight Show with Jay Leno	22	77	Billy Crystal/Garth Brooks/Final Show	8.2	3
	Bu Gece	None	None	Ibrahim Selim Ile Bu Gece #63	9.4	4
	Bu Gece	6	1	Hazir miyiz? Alkislarla Kerem Bürsin	9.1	4
	Dora the Explorer	4	19	Dora's World Adventure!	9.8	5
	Bu Gece	None	None	Ibrahim Selim Ile Bu Gece #79	8.6	4
	The View	27	129	Sydney Sweeney/Chef Jose Andres	7.9	3
	Bu Gece	None	None	Ibrahim Selim Ile Bu Gece #90	8.5	4
	The View	17	148	Guest Co-Hostess Candace Cameron Bure/Phil McGraw/Kelly Reilly and Vanessa Redgrave	7.8	3
	The Powers of Matthew Star	1	21	Swords and Quests	8.0	4

Truncated to [displaylimit](#) of 10.